



Programmieren

Falk Morgenstern

Leon Johann Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.
Keine Weitergabe.



Programmieren

5. Vererbung

5.1 Erben von einer konkreten Klasse

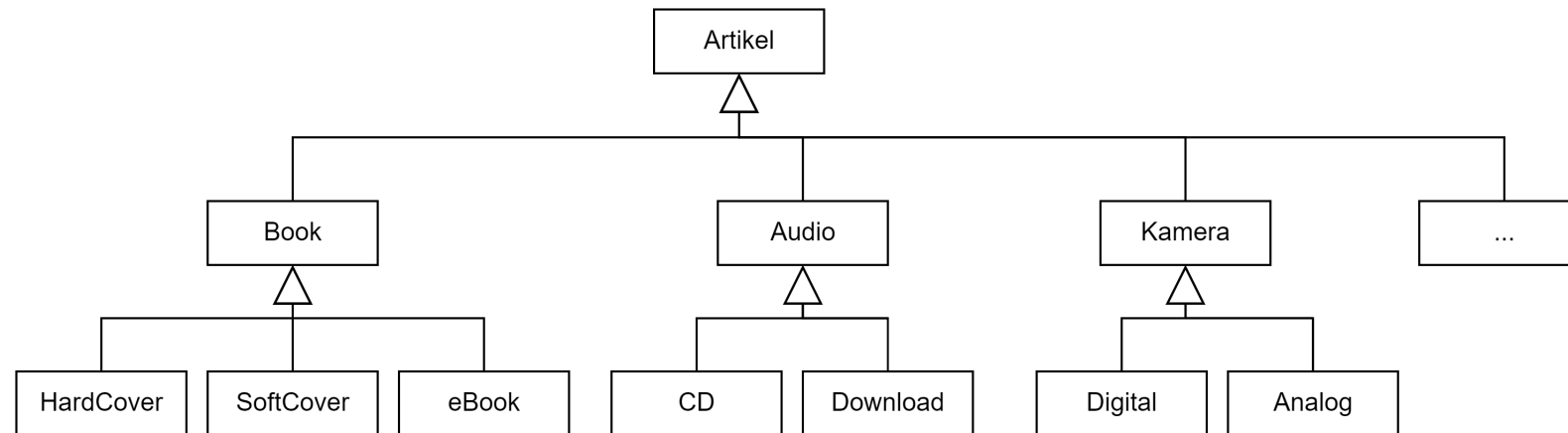


Vererbung

- Klassen modellieren Dinge der realen Welt z.B. Personen, Konten, Autos
- Varianten von Dingen lassen sich hierarchisch gliedern bzw. klassifizieren
- Tiere:
 - Fische, Reptilien, Säugetiere, . . .
 - Säugetiere: Raubtiere, Nagetiere, Huftiere, . . .
 - Konkrete Tierarten wie Katze, Hund, Pferd
- Artikel in einem Web-Shop:
 - Buch, Audio, Kamera . .
 - Buch: Hardcover, SoftCover, eBook . . .
 - Audio: CD, Cassette



Hierarchische Gliederung: Beispiel Webshop



- Was bringt uns die hierarchische Gliederung?
 - eBook hat alle Eigenschaften eines Buchs und zusätzlich eigene Eigenschaften (z.B. Informationen über unterstützte Reader)
 - Buch hat alle Eigenschaften eines Artikels und zusätzlich eigene Eigenschaften (z.B. Autor)
 - CD und Download lassen sich als Audio behandeln.
 - Buch, Audio, Kamera lassen sich als Artikel behandeln.



Abstraktion und Spezialisierung

- Programme sollen in der Lage sein, mit verschiedenen Varianten eines Objekts zu arbeiten (Abstraktion).
- Die Varianten (Objekte verschiedener Unterklassen) sind gleich zu behandeln.
- Jede spezielle Klasse bringt zusätzliche eigene Eigenschaften mit (Spezialisierung).
- Vorteil: Im Webshop kann einfach eine neue Gruppe von Artikeln z.B. Video, eingefügt werden, ohne die anderen Artikel-Gruppen und die Klasse Artikel zu ändern.



Spezialisierung Beispiel Webshop

- Informationen zu Büchern:
 - Artikelnummer
 - Preis
 - Verfasser
 - Titel
 - Verlag
 - Erscheinungsjahr
- Informationen zu CDs:
 - Artikelnummer
 - Preis
 - Interpret
 - Liste von Musikstücken



Spezialisierung Beispiel Webshop

- Kundensicht auf Warenkorb im Webshop:
 - Warenkorb enthält alle ausgewählten Artikel mit Namen und Preis
 - Informationen über ausgewählte Artikel abrufbar
 - Keine Unterscheidung zwischen Artikeln (Bücher, CDs, ...) im Warenkorb



Spezialisierung Beispiel Webshop

- Implementierung mit Klasse `Article`, die alle Attribute und Methoden enthält, die für Bücher, CDs, etc. gemeinsam gelten sollen

```
public class Article {  
  
    private int code; // article number  
    private String label; // short description  
    private BigDecimal price;  
  
    public Article (int code, String label,  
                   BigDecimal price) {  
        /* ... */  
    }  
  
    public void showInfo() { /* ... */ }  
  
    public String getArticleLine() { /* ... */ }  
}
```





Vererbung: Klasse Book als Spezialisierung von Article

- Bücher (und auch CDs) sind Spezialisierungen von Artikeln
 - Haben dieselben Methoden und Attribute wie Artikel
 - Haben noch weitere Attribute und Methoden
- Terminologie:
 - Article ist **Oberklasse** von Book.
 - Book ist **Unterklasse** von Article.
 - Book **erbt** alle Attribute und Methoden von Article.
 - Book **überschreibt** die Methode `showInfo()` von Article.

```
public class Book extends Article {  
  
    private String author, title, publisher;  
    private int year;  
  
    Book(int code, String author, String title,  
         BigDecimal price)  
    {  
        super(code, author+title, price);  
    }  
  
    public void showInfo() { /* ... */ }  
  
}
```



Vererbung: Klasse CD als Spezialisierung von Article und weitere Unterklassen

- Wir können weitere Unterklassen von Article bilden, z.B. CD
- Wir könne weitere Unterklassen von Book (Unterklasse von Article) bilden, z.B. Hardcover

```
public class CD extends Article {  
  
    private String artist;  
    private String[] songs;  
  
    public CD(int code, String label,  
              BigDecimal price) {  
        super(code, label, price);  
    }  
}
```



```
public class Hardcover extends Book {  
    Hardcover(int code, String author,  
              String title, BigDecimal price) {  
        super(code, author, title, price);  
    }  
}
```





Überschreiben von Methoden

- Book, CD erben unverändert die Methode `getArticleLine()` von `Article`
 - `getArticleLine()` stellt aus `code`, `label`, `price` eine Zeile für den Warenkorb zusammen
- Book, CD erben die Methode `showInfo()` von `Article`
 - `showInfo()` zeigt Informationen über den Artikel in einem Fenster an
 - Da es über Bücher und CDs mehr Informationen gibt, als in `Article` vorgesehen, wird die Methode `showInfo()` überschrieben.
 - Überschreiben meint: mit gleicher Signatur neu implementieren.



super-Aufrufe

- Eine Methode, die eine Methode der Oberklasse überschreibt, kann die Oberklassen-Methode trotzdem verwenden.
- Beispiel: `showInfo()` aus `Book` will `showInfo()` aus `Article` verwenden, um die Attribute `code`, `label`, `price` darzustellen. Anschließend werden weitere Informationen hinzugefügt, die spezifisch für die Klasse `Book` sind.
- Aufruf einer Methode der Oberklasse durch Verwendung von **super**.



Beispiel: super-Aufrufe in Book

- Mittels `super` wird die **Methode** der Oberklasse aufgerufen
- Im **Konstruktor** muss der Aufruf des Konstruktors der Oberklasse die erste Anweisung sein, sonst wird der default-Konstruktor aufgerufen (wenn vorhanden)

```
public class Book extends Article {  
  
    private String author, title, publisher;  
    private int year;  
  
    Book(int code, String author, String title,  
         int price) {  
        super(code, author+title, price);  
    }  
  
    public void showInfo() {  
        super.showInfo(); // show article info  
        /* ... show author, title, publisher ... */  
    }  
  
}
```



Kompatibilität zwischen Ober- und Unterklasse

- Was nützt uns Vererbung?
 - Unterklasse ist mit der Oberklasse kompatibel.
 - D.h. jedes Programm, das in der Lage ist, mit Objekten der Oberklasse zu arbeiten, kann automatisch auch mit Objekten der Unterklasse arbeiten.
 - Ist-Beziehung („is-a“) zwischen Unter- und Oberklasse



Kompatibilität zwischen Ober- und Unterklasse

Beispiel Webshop

- Wir können ein Programm schreiben, das mit Artikeln (Klasse `Article`) arbeitet:
 - Artikel in den Warenkorb legen, Warenkorb anzeigen, ausdrucken
 - Informationen über einen Artikel im Warenkorb anzeigen
- Später können wir konkrete Artikel wie Bücher, CDs implementieren.
 - Unser Programm kann sofort mit ihnen arbeiten, da sie von `Article` abgeleitet sind.
 - Neue Artikelgruppen, wie „Video“ oder „Computerspiele“ können hinzugefügt werden, indem man sie als Unterklassen von `Article` realisiert. Das bestehende Programm arbeitet ohne Änderungen mit den neuen Artikelgruppen.



Ist-Beziehung zwischen Ober- und Unterklasse

- Zwischen der Klasse Artikel und den Unterklassen besteht die folgende Ist-Beziehung bzw. **Teilmengen-Beziehung**
 - Jedes Objekt der Unterklasse **ist** Objekt der Oberklasse.
 - CD **ist** Article
 - Book **ist** Article
 - Hardcover **ist** Book
 - Hardcover **ist** Article



Zuweisungen zwischen Objekten aus Ober- und Unterklassen

- Einem Objekt der Oberklasse kann man ein Objekt der Unterklasse zuweisen.
- Book-Objekt kann `Article`-Variablen zugewiesen werden
- Es können nur Attribute und Methoden von `Article` verwendet werden.
- Attribute von Book wie `author`, `title` und Methoden können nicht referenziert werden.
- Wir können mit `instanceof` testen, ob `a` auf ein Book-Objekt zeigt und mit einem `Cast (...)` den Typ von `a` nach Book wandeln.

```
Article a = new Book(4711, "Müller", "Poems", 17);  
  
if (a instanceof Book) { // Laufzeittest  
    Book book = (Book)a; // Typumwandlung (kann fehlschlagen)  
}
```





Programmieren

5. Vererbung

5.2 Statische und dynamische Typen



Dynamische Bindung

- Zuweisung eines Objektes einer Unterklasse an eine Referenzvariable der Oberklasse:

```
Article a = new Book(...);  
a.showInfo();
```



- Welche Methode `showInfo()` wird benutzt?
- `showInfo()` aus `Article` oder `showInfo()` aus `Book`?
- Woher weiß die JVM, welche Implementierung verwendet werden soll?
- Es wird immer die Methode aufgerufen, die zum **dynamischen Typ** des Objekts gehört. Die JVM entscheidet erst **zur Laufzeit**, was der dynamische Typ ist, und welche Methode sie daher auszuführen hat.



Statischer und dynamische Typ

- Der Typ einer Variablen, wie er bei der Deklaration im Quelltext angegeben wird, wird **statischer Typ** genannt.
- Der statische Typ ist dem Compiler bekannt und kann sich während der Laufzeit nicht ändern.
- Die JVM stellt bei jedem Methodenaufruf zur Laufzeit den aktuellen Typ des momentan zugewiesenen Zielobjekts fest. Dieser Typ wird **dynamischer Typ** genannt und kann vom statischen Typen abweichen.
- Der Compiler kennt den dynamischen Typen nicht, prüft aber, dass er mit der Oberklasse kompatibel sein wird.

```
Article a = new Book(...);
```

statischer Typ dynamischer Typ



Dynamische Bindung

- Die JVM stellt bei jedem Methodenaufruf zur Laufzeit den aktuellen Typ des momentan zugewiesenen Zielobjekts fest und ruft dann dessen Methode auf.
- Das heißt, dass zur Compilezeit noch nicht festgelegt werden muss, welche Klasse die Methode bereitstellen wird.
- Definition **Dynamische Bindung**
 - Zuordnung eines Methodenaufrufs zu einem konkreten Methodenrumpf, geschieht erst zur Laufzeit.

```
Article a = new Book(...);
```

statischer Typ dynamischer Typ



Statische und dynamische Typen

- Die Variable `a` hat den **statischen Typ** `Article`.
 - Der statische Typ bestimmt, welche Attribute und Methoden **ansprechbar** sind.
- Die Variable `a` hat den **dynamischen Typ** `Book`.
 - Der dynamische Typ bestimmt, welche Methoden**implementierungen** aufgerufen werden.
 - Kann sich mit jeder neuen Zuweisung ändern.

```
Article a = new Book(...);
```

statischer Typ dynamischer Typ



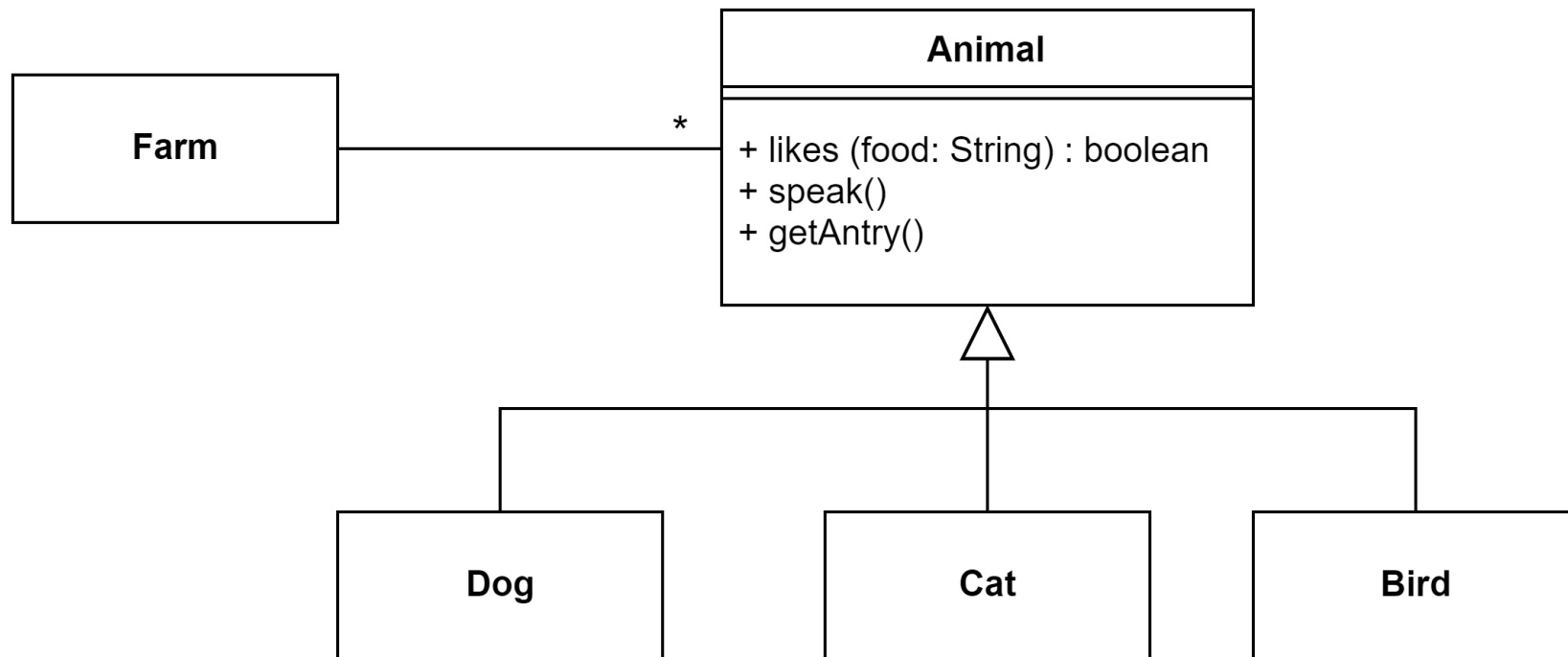
Beispiel: Webshop

- `showInfo()` ist eine Methode in der Klasse `Article`
- Damit besitzt jedes Objekt einer abgeleiteten Klasse diese Methode (auch `Book`, `CD`, ...)
- Immer wenn `showInfo()` auf einem Objekt aufgerufen wird, dessen dynamischer Typ eine Unterklasse von `Article` ist, wird die in der Unterklasse gültige und ggf. dort implementierte (überschriebene) Version verwendet.

```
Article a = new Book(...);
```

statischer Typ dynamischer Typ

Beispiel: Tierfarm





Beispiel: Tierfarm

- Jedes Tier hat die folgenden Möglichkeiten, sich zu „äußern“
 - `likes(food)`: Gefallen am Futter zeigen (das Tier mag das Futter)
 - `speak()`: Laut(e) von sich geben
 - `getAngry()`: zornig werden

```
public class Animal {  
  
    public boolean likes(String food) {  
        return false;  
    }  
    public void speak() {  
        System.out.println("???");  
    }  
    public void getAngry() { speak(); }  
}
```





Beispiel: Tierfarm

```
public class Dog extends Animal {  
    public boolean likes(String food) { return food.equals("bones"); }  
    public void speak() { System.out.println("woof"); }  
    public void getAngry() { System.out.println("bite"); }  
}
```



```
public class Cat extends Animal {  
    public boolean likes(String food) { return food.equals("milk"); }  
    public void speak() { System.out.println("meow"); }  
    public void getAngry() { System.out.println("scrape"); }  
}
```



```
public class Bird extends Animal {  
    public boolean likes(String food) { return food.equals("corn"); }  
    public void speak() { System.out.println("beep"); }  
}
```





Beispiel: Tierfarm

- Klasse Farm verwaltet ein Array von Objekten der Klasse Animal

```
public class Farm {  
  
    private Animal[] animals;  
  
    public void feed(String food) {  
        for (Animal animal : animals) {  
            if (animal.likes(food)) {  
                animal.speak();  
            } else {  
                animal.getAngry();  
            }  
        }  
    }  
}
```





Anwenden der Methode „feed“

- Wir definieren ein Objekt `farm` der Klasse `Farm`, das mit Tieren der folgenden Liste von Typen instanziiert ist:
 - `Cat`, `Dog`, `Cat`, `Bird`
- Wir füttern die Tiere mit Milch:
 - `farm.feed("milk");`
- Das führt zu folgendem Ergebnis:
 - `meow bite meow beep`
- Frage: Wie ist das Ergebnis entstanden?



Dynamische Bindung

- **Dynamische Bindung** erfolgt
 - in einer **Vererbungsbeziehung** (bzw. -hierarchie)
 - im Falle einer in Methode, die in der (den) abgeleiteten Klasse(n) **überschrieben** wurde
- Laufzeitumgebung wählt erst zur Laufzeit anhand des **Objektyps** (anstelle des Referenztyps) aus, welche (überschriebene) Methodenimplementierung verwendet wird, passend zum tatsächlichen Typ des aufrufenden Objekts
- Je nach dem tatsächlichen Objekttyp nimmt eine Methode daher „unterschiedliche Gestalt“ an: **Polymorphie**
- Private, statische und finale Methoden werden statisch gebunden
 - Private, statische und finale Methoden können nicht überschrieben werden
 - Daher gibt es in diesem Fall keine dynamische Bindung
- Überladene Methoden werden statisch gebunden



Dynamischer Polymorphismus

- **Polymorph** (griech.): vielgestaltig. In der Objektorientierung die Idee, dass einer Variable Objekte unterschiedlicher dynamischer Typen zugewiesen werden können und dadurch beim Aufruf derselben Operation unterschiedliches Verhalten gezeigt wird.
- Eine Referenzvariable vom Typ A kann auf Objekte aller Klassen verweisen, die Unterklassen von A sind.
- Analog für Interfaces (siehe später).
- Die Variable ist in dem Sinne polymorph (vielgestaltig), dass sie Objekte vieler verschiedener Typen aufnehmen kann.
- Vorteil: Solange im Code nur Methoden der Oberklasse genutzt werden, benötigt der Entwickler keinerlei Wissen über die unter der Oberklasse liegende Vererbungshierarchie.



Regeln zum Methodenaufruf

1. Der **dynamische Typ** einer Instanzvariablen *t* bestimmt, **in welcher Klasse** nach der entsprechenden Methode gesucht wird.
2. Hat eine Methode **Parameter**, so werden die aktuellen Parameter gemäß ihres **statischen Typs** im Methodenkopf behandelt.
3. Gibt es mehrere überschriebene Methoden, wird diejenige ausgewählt, deren Signatur am besten (= engsten) zu den statischen Typen der vorliegenden aktuellen Parameter passt. Dabei wird - ausgehend von der Klasse selbst - schrittweise in der jeweils nächsten Oberklasse nach Methoden mit passender Signatur gesucht.



Übung: Dynamische Typen I

```
public class Top {  
    public void meth(Top t)    {System.out.println("TopTop ");}  
    public void meth(Middle m) {System.out.println("TopMid ");}  
}
```



```
public class Middle extends Top {  
    public void meth(Object o) {System.out.println("MidObj ");}  
    public void meth(Top t)    {System.out.println("MidTop ");}  
    public void meth(Middle m) {System.out.println("MidMid ");}  
}
```



```
public class TypeTest {  
    public static void main(String[] args){  
        Top top = new Top();  
        Top tm = new Middle();  
        Middle mid = new Middle();  
        top.meth(mid);  
        tm.meth(top);  
        tm.meth(mid);  
    }  
}
```





Übung: Dynamische Typen II

```
public class Top {  
    public void meth(Top t)    {System.out.println("TopTop ");}  
    public void meth(Middle m) {System.out.println("TopMid ");}  
}
```



```
public class Middle extends Top {  
    public void meth(Object o) {System.out.println("MidObj ");}  
    public void meth(Top t)    {System.out.println("MidTop ");}  
    public void meth(Middle m) {System.out.println("MidMid ");}  
}
```



```
public class TypeTest {  
    public static void main(String[] args){  
        Top top = new Top();  
        Top tm  = new Middle();  
        Object obj = new Middle();  
        top.meth(tm);  
        tm.meth(tm);  
        tm.meth(obj);  
    } }  
}
```





Übung: Dynamische Typen III

```
public class Top {  
    public void meth(Top t)    {System.out.println("TopTop ");}  
    public void meth(Middle m) {System.out.println("TopMid ");}  
}
```



```
public class Middle extends Top {  
    public void meth(Object o) {System.out.println("MidObj ");}  
    public void meth(Top t)    {System.out.println("MidTop ");}  
    public void meth(Middle m) {System.out.println("MidMid ");}  
}
```



```
public class Bottom extends Middle{  
    public void meth(Object o) {System.out.println("BotObj ");}  
    public void meth(Bottom b) {System.out.println("BotBot ");}  
}
```



```
public class TypeTest {  
    public static void main(String[] args){  
        Top top = new Top();  
        Object obj = new Middle();  
        Middle mb = new Bottom();  
        mb.meth(top);  
        mb.meth(obj);  
        mb.meth(mb);} }
```

Übung: Dynamische Typen IV - Ausgaben



```
// Dynamische Typen I
```

```
TopMid
```

```
MidTop
```

```
MidMid
```

```
// Dynamische Typen II
```

```
TopTop
```

```
MidTop
```

```
Compilerfehler: In der Klasse Top gibt es keine  
Methode meth(Object o)
```

```
// Dynamische Typen III
```

```
MidTop
```

```
BotObj
```

```
MidMid
```



@Override

- Wenn eine Methode in einer abgeleiteten Klasse überschrieben wird, sollte dort die @Override-Annotation verwendet werden

```
@Override
public void showInfo() {
    super.showInfo(); // show general article info
    /* ... show author, title, publisher ... */
}
```



- Damit signalisiert der Programmierer dem Compiler, dass die folgende Methode eine entsprechende Methode in der Basisklasse überschreiben soll
- Vorteil: Wenn es diese Methode in der Basisklasse nicht gibt (z.B. wegen eines Programmierfehlers oder weil sich die Methode in der Basisklasse geändert hat), generiert der Compiler eine Fehlermeldung



Programmieren

5. Vererbung

5.3 Abstrakte Klassen



Abstrakte Methoden

- Nutzung von Vererbung im Beispiel Tierfarm wirkt künstlich, da keine der Methoden in der Klasse Animal sinnvoll implementiert werden konnte.
- z.B. wurde `speak()` in jeder Unterklasse (Cat, Dog, Bird) überschrieben.
- In manchen Fällen lässt sich in der Oberklasse keine sinnvolle Implementierung einer Methode angeben, obwohl in jeder Unterklasse eine Implementierung vorhanden sein soll.
- In solchen Fällen kann die Implementierung dieser Methode in der Oberklasse weggelassen werden; die Methode wird dann zu einer **abstrakten Methode**



Abstrakte Methoden, abstrakte Klassen

- Eine Methode, für die keine Implementierung angegeben ist, bezeichnet man als **abstrakte Methode**.
- Schlüsselwort für abstrakte Methoden: **abstract**
- Man gibt nur die Signatur an, aber keinen Code.
- Eine Klasse, die mindestens eine abstrakte Methode enthält, bezeichnet man als **abstrakte Klasse**.



Beispiel: Abstrakte Klasse

```
public abstract class Animal {  
    public boolean likes(String food) {return false;}  
  
    public abstract void speak();  
  
    public void getAngry() {speak();}  
}
```



- Konkrete Unterklassen einer abstrakten Klasse müssen alle abstrakten Methoden implementieren oder selbst wieder als abstrakt gekennzeichnet werden.



Deklaration von Variablen

- Da abstrakte Klassen keine vollständige Implementierung besitzen, kann und darf man keine Objekte der Klasse erzeugen.
- Man kann aber Objekte von (nicht abstrakten) Unterklassen erzeugen. Daher kann die Oberklasse als statischer Typ vorkommen.
- Der dynamische Typ muss der Typ einer Unterklasse sein, da nur sie alle Methoden implementiert.

```
Animal a; // erlaubt   
Animal b = new Animal(); // Compilerfehler  
Animal c = new Bird();  
c.speak(); // dynamische Bindung
```



Zweck abstrakter Klassen

- Abstrakte Methoden (und damit abstrakte Klassen) werden eingesetzt, wenn kein sinnvolles gemeinsames Verhalten für die Methode implementiert werden kann, wenn die Methode aber trotzdem in allen Unterklassen zur Verfügung stehen soll.
- Abstrakte Klassen dienen dazu, die Schnittstellen künftiger Unterklassen festzulegen.
- Eine abstrakte Methode legt die Signatur der Methode der Unterklasse fest.
- Unterklassen, von denen Objekte erzeugt werden sollen, müssen die abstrakten Methoden implementieren.



Programmieren

5. Vererbung

5.4 Interfaces

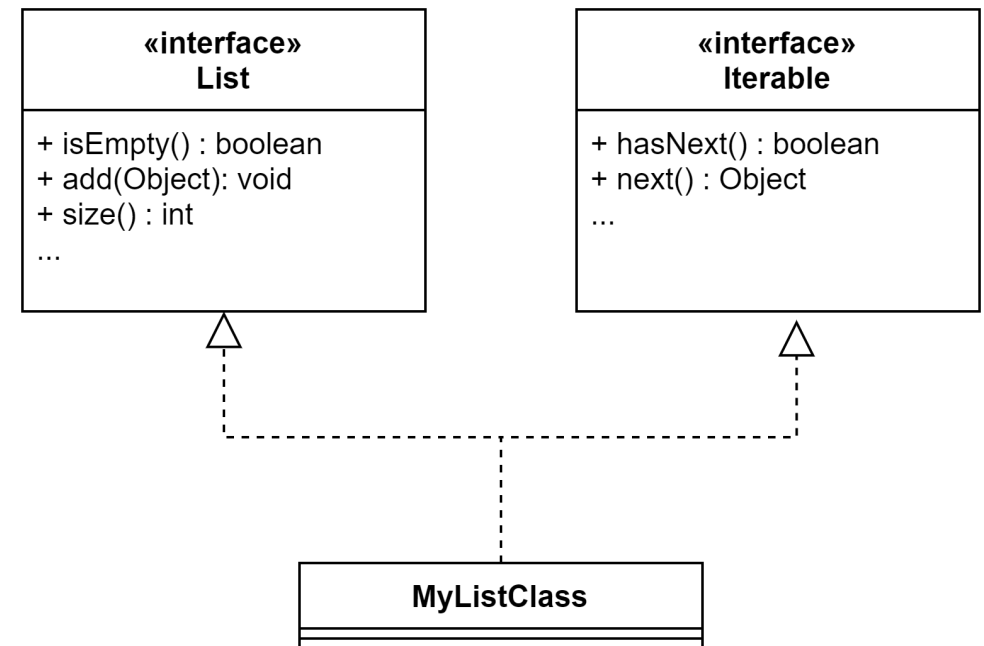


Motivation

- Angenommen (Java < 8), eine Klasse enthält **nur abstrakte Methoden** $\rightsquigarrow$ führt zum Konzept des Interface (Schnittstelle)
- Ein Interface ist eine Sammlung von (abstrakten) Methoden (und Konstanten)
- Eine Klasse implementiert ein Interface, wenn sie Implementierungen für alle Methoden des Interface bereitstellt
- Jede konkrete Klasse, die ein Interface implementiert, stellt eine Implementierung für alle (abstrakten) Methoden des Interface bereit
- Damit gibt die Schnittstelle (= Interface) vor, was alle implementierenden Klassen leisten können / wie sie benutzt werden können
- Bild: Steckbuchsen an einem Gerät, wenn der Steck passt, müsste es funktionieren..

Motivation

- In Java kann eine Klasse nur von genau einer Klasse erben. Mehrfachvererbung ist ausgeschlossen. Eine Klasse kann aber mehrere Interfaces implementieren.
- Damit erreicht man, dass eine Klasse zu verschiedenen Datentypen (nämlich den Interface-Typen) kompatibel ist, d.h. sie unterstützt alle Methoden, die benötigt werden.





Interfaces

- Interfaces sind (reine) Schnittstellen in Java
 - Interfaces geben vor, wie eine Klasse genutzt werden kann.
- Darf keine Implementierungsdetails enthalten
 - Keine Klassen- oder Instanzvariablen, keine Konstruktoren
- **Beinhaltet Methodensignaturen zur Beschreibung, was eine Klasse leisten soll**, d.h. keine Methode ist implementiert (in der ursprünglichen Interface-Version vor Java 8)!
- Interfaces können Konstanten enthalten
 - Sind auch ohne Schlüsselwort “final” konstant und werden vererbt
- Seit Java 8 dürfen Interfaces auch implementierte Methoden enthalten
 - “Default-Methoden” (Schlüsselwort: default); werden vererbt
- Seit Java 8 dürfen Interfaces auch statische Methoden enthalten
 - Diese werden nicht vererbt!
- Man sagt: Eine Klasse implementiert ein Interface
 - Schlüsselwort: `implements`



Beispiel: Interface

```
public interface Animal {  
  
    boolean likes(String food);  
    void speak();  
    void getAngry();  
  
}
```



- Die Methoden eines Interface sind öffentlich und abstract (`public`, `abstract`)
- (letzteres gilt nicht für Default- und statische Methoden).
- Alle Attribute sind immer Konstanten (`public static final`).



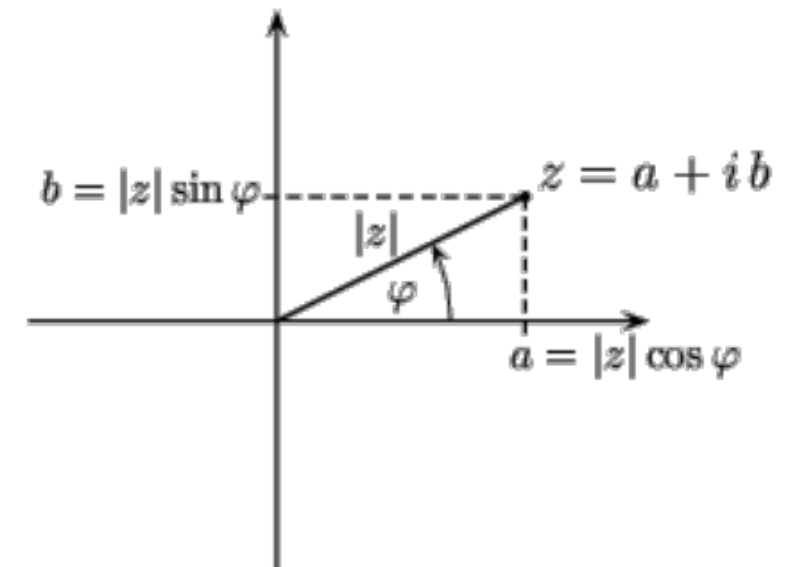
Beispiel: Interface

- Damit eine Klasse das Interface `Animal` realisiert, muss es das Interface `Animal` implementieren, d.h. Implementierungen für alle Methoden von `Animal` bereitstellen.
- `Bird` implementiert `Animal`.
- Die Schnittstelle ist in diesem Beispiel `Animal`.
- `Bird` ist unsere Implementierung.

```
public class Bird implements Animal {  
  
    public boolean likes(String food) {  
        return food.equals("corn");  
    }  
  
    public void speak() {  
        System.out.println("beep");  
    }  
  
    public void getAngry() {  
        System.out.println("beep");  
    }  
  
}
```

Beispiel: Komplexe Zahlen

- Komplexe Zahlen können durch kartesische oder durch Polardarstellung angegeben werden.
- In kartesischer Darstellung werden die Koordinaten auf der X- und Y-Achse angegeben.
- In Polardarstellung wird die Distanz zwischen Punkt und Koordinaten-ursprung sowie der Winkel von der Abszisse zum Punkt angegeben.



© mathe-online.fernuni-hagen.de/MIB/HTML/node38.html



Beispiel: Komplexe Zahlen

- Komplexe Zahlen können auf verschiedene Weise charakterisiert werden:
- Realteil Betrag
- Imaginärteil Phase
- Um komplexe Zahlen zu unterstützen, muss eine Klasse (Rechen-)Methoden bereitstellen.
- Damit bietet sich eine Kapselung der möglichen Implementierungen über eine Schnittstelle an!



Beispiel: Komplexe Zahlen

```
public interface Complex {  
  
    public double getReal();  
    public double getImag();  
    public double getDistance();  
    public double getPhase();  
  
}
```



- Eine Klasse implementiert komplexe Zahlen, wenn sie Methoden für den Realteil, den Imaginärteil, die Distanz zum Ursprung und die Phase bereitstellt.



Beispiel: Komplexe Zahlen

```
public interface Complex {  
  
    public double getReal();  
    public double getImag();  
    public double getDistance();  
    public double getPhase();  
    public Complex add(Complex c);  
    public Complex mult(Complex c);  
}
```



- Zusätzlich sollen (stellvertretend) die Grundrechenarten Addition und Multiplikation unterstützt werden



Beispiel: Komplexe Zahlen

- Als nächstes erstellen wir die Klasse `Cartesian`. Sie soll komplexe Zahlen realisieren, d.h. sie soll eine Implementierung der Schnittstelle `Complex` sein.
- `Java: Cartesian implements Complex`
- Eine Klasse kann weitere Methoden implementieren, die über die Methoden der Schnittstelle hinausgehen. Aber alle Methoden, die eine Schnittstelle vorgibt, müssen implementiert werden!
- Die `Cartesian`-Klasse benötigt einen Konstruktor, der die Real- und Imaginär-Koordinate übergibt. Ein Konstruktor kann nicht vom Interface vorgegeben werden.



Beispiel: Komplexe Zahlen

```
public class Cartesian implements Complex {  
    private double real;  
    private double imag;  
  
    public Cartesian(double real, double imag) {  
        this.real = real;  
        this.imag = imag;  
    }  
  
    public double getReal() { return real; }  
    public double getImag() { return imag; }  
  
    public double getDistance() { return Math.hypot(real, imag); }  
    public double getPhase() { return Math.atan2(imag, real); }  
  
    public Complex add(Complex that) { return null; }  
    public Complex mult(Complex that) { return null; }  
}
```





Beispiel: Komplexe Zahlen

- Die Klasse `Polar` implementiert ebenfalls das Interface `Complex`.
- Auch `Polar` realisiert komplexe Zahlen.
- `Polar` arbeitet im Unterschied zu `Cartesian` mit Polarkoordinaten.



Beispiel: Komplexe Zahlen

```
public class Polar implements Complex {  
    private double distance;  
    private double phase;  
  
    public Polar(double d, double p) {  
        this.distance = d;  
        this.phase = p;  
    }  
  
    public double getReal() {return distance*Math.cos(phase);}   
    public double getImag() {return distance*Math.sin(phase);}   
  
    public double getDistance() {return distance;}  
    public double getPhase() {return phase;}  
  
    public Complex add(Complex that) { return null; }  
    public Complex mult(Complex that) { return null; }  
}
```



Beispiel: Komplexe Zahlen

- Die beiden Darstellungen für komplexe Zahlen können wir jetzt wie folgt benutzen:

```
public static void main(String[] args) {  
    Complex complexNumber1 = new Cartesian(1,2);  
    System.out.println(complexNumber1.getDistance());  
  
    Complex complexNumber2 = new Polar(4,Math.PI/2);  
    System.out.println(complexNumber2.getDistance());  
}
```





Beispiel: Komplexe Zahlen

- Wir können Referenzvariablen vom Typ `Complex` Objekte vom Typ `Cartesian` oder `Polar` zuweisen, da beide Klassen das Interface `Complex` implementieren.
- Wir können auch eine Referenzvariable vom Typ `Complex` erstellen und ihr erst eine kartesische Darstellung zuweisen und dann eine Polardarstellung:

```
public static void main(String[] args) {  
    Complex complexNumber = new Cartesian(1,2);  
    System.out.println(complexNumber.getDistance());  
  
    complexNumber = new Polar(4,Math.PI/2);  
    System.out.println(complexNumber.getDistance());  
}
```





Beispiel: Komplexe Zahlen

- Die Entscheidung, ein `Cartesian`- oder `Polar`-Objekt zu verwenden, kann zur Laufzeit vom Nutzer getroffen werden
- Woher weiß die JVM, welche Implementierung verwendet wird, wenn es von der Eingabe des Nutzers abhängt?



```
public void foo() {  
    Scanner scan = new Scanner(System.in);  
    Complex complexNumber;  
  
    if (scan.next() == "a") {  
        complexNumber = new Cartesian(1, 2);  
    } else {  
        complexNumber = new Polar(1, Math.PI / 2);  
    }  
}
```



Dynamische Bindung

```
Complex variable = new Polar (2,85);
```

Statischer Typ

Dynamischer Typ

- Es wird erst zur Laufzeit von der JVM entschieden, von welcher Klasse eine Instanz erzeugt wird.
- Wie bei Vererbung, stellt die JVM bei jedem Methodenaufruf zur Laufzeit den dynamischen Typ (aktueller Typ des momentan zugewiesenen Zielobjekts) fest und ruft dann dessen Methode auf (dynamische Bindung).
- Der Compiler kennt den dynamischen Typen nicht, prüft aber, dass er mit dem Interface (oder der Oberklasse) kompatibel sein wird.
- Bei Interfaces bedeutet das, dass eine Variable den statischen Typ des Interface bekommt. Da die Methoden in allen Implementierungen des Interfaces eine Realisierung haben müssen, weiß der Compiler, welche Methoden auf dem referenzierten Objekt aufgerufen werden können.



Interfaces als Parametertyp

- Wir können Interface-Typen auch als Parametertyp verwenden.
- In diesem Fall können wir nur die Methoden benutzen, die im Interface definiert sind.
- Beim Aufruf einer Methode mit einem Parameter von einem Interface-Typen muss der aktuelle Parameter dieses Interface implementieren.
- Als Beispiel erweitern wir `Cartesian` und `Polar` um einen Konstruktor, der eine komplexe Zahl erhält und sie in die kartesische bzw. Polardarstellung umwandelt.



Beispiel: Komplexe Zahlen

```
public class Polar implements Complex {  
  
    public Polar (Complex c) {  
        this.distance = c.getDistance();  
        this.phase = c.getPhase();  
    }  
  
    // ...
```



```
public class Cartesian implements Complex {  
  
    public Cartesian(Complex c) {  
        this.real = c.getReal();  
        this.imag = c.getImag();  
    }  
  
    // ...
```





Complex als Parameter

- Die Getter und Setter sind im Interface vorgegeben.
- Da nur dynamische Typen, die das Interface `Complex` implementieren, als Parameter auftreten können, wird es auch immer eine Implementierung für die Methoden geben.
- Den Konstruktoren kann - zur Laufzeit - ein Objekt der Klasse `Cartesian` oder eines der Klasse `Polar` mitgegeben werden. Beide beinhalten Implementierungen für die Methoden, die im Konstruktor aufgerufen werden.
- Die Nutzung des Interfaces schafft eine Grenze zwischen den Methoden, die der Nutzer komplexer Zahlen verwenden soll, und der Implementierung, die intern bleibt → Kapselung



Interfaces als Rückgabetypen

- Analog können wir `Complex` als Typ des Rückgabewertes verwenden.



```
public class Cartesian implements Complex {  
  
    public Complex add(Complex c) {  
        double r = real + c.getReal();  
        double i = imag + c.getImag();  
        return new Cartesian(r, i);  
    }  
  
    // ...  
}
```



Complex als Rückgabewert

- In diesem Interface sind keine Methoden und Konstruktoren implementiert, da es ja eine Schnittstelle ist.
- Konsequenz: Es können keine Objekte des Typen `Complex` erzeugt werden.
- Wir müssen also eine Klasse benutzen, die `Complex` implementiert.
- Der dynamische Typ stellt Implementierungen für alle verwendeten Methoden und einen Konstruktor bereit.

```
public Complex add(Complex c) {  
    double r = real + c.getReal();  
    double i = imag + c.getImag();  
    return new Complex(r, i);  
}  
  
// Comilerfehler! ...
```





Complex als Rückgabewert

- Es ist auch möglich, als Rückgabebetyp nicht Complex anzugeben, sondern eine Klasse, die Complex implementiert.
- Die Methode gilt damit immer noch als Implementierung der Interface-Methode.
- Wichtig ist: Der dynamische Typ des Rückgabewerts muss eine Implementierungsklasse sein, der statische Typ des Rückgabewerts darf auch ein Interface sein.

```
public class Cartesian implements Complex {  
  
    public Cartesian add(Complex c) {  
        double r = real + c.getReal();  
        double i = imag + c.getImag();  
        return new Cartesian(r, i);  
    }  
  
    // ...  
}
```



Nutzungsbeispiel: ReadWrite

- Welche Vorteile bieten Interfaces?
 - Interfaces entkoppeln die Nutzung von Funktionalität von Änderungen der Implementierung, d.h. Nutzer einer Schnittstelle müssen ihren Code nicht ändern, wenn sich Implementierungsinterna ändern.
- Das Interface `ReadWrite` beinhaltet die beiden Methoden `read` und `write`. Neben abstrakten Methoden kann ein Interface auch Konstanten enthalten.

```
public interface ReadWrite {  
    String dataBasePassword = "alligator3";  
    String read();  
    void write(String dataToWrite);  
}
```





Nutzungsbeispiel: ReadWrite

```
public class Cloud implements ReadWrite {  
    public String read() {  
        return readDataFromCloud(databasePassword);  
    }  
    public void write(String data) {  
        writeDataToCloud(databasePassword, data);  
    }  
}
```



- Die Klasse Cloud und HardDrive implementieren beide das Interface ReadWrite

```
public class HardDrive implements ReadWrite {  
    public String read() {  
        return readDataFromHardDrive();  
    }  
    public void write(String data) {  
        writeDataToHDD(data);  
    }  
}
```





Nutzungsbeispiel: ReadWrite

- Wenn jetzt Daten, die bisher auf der Festplatte gespeichert sind, nun in der Cloud gespeichert werden sollen, muss nur der dynamische Typ ausgetauscht werden.
- Der statische Typ ReadWrite bleibt unverändert. Die Methode write ist auf jeden Fall vorhanden.
- Der dynamische Typ bestimmt, wo die Daten gespeichert werden, da der Code des dynamischen Typen ausgeführt wird.

```
ReadWrite save = new Cloud();  
String data = "01000010101101";  
save.write(data);
```





Vererbung und Interfaces

UML

- Klasse A wird von Klasse B abgeleitet

```
public class A extends B { /* ... */ }
```



- Klasse C implementiert Interface D

```
public class C implements D { /* ... */ }
```



- Interface E erweitert Interface D

```
public interface E extends D { /* ... */ }
```





Vererbung und Interfaces

- Die Implementierung von Interfaces kann mit Vererbung kombiniert werden.
- Als Beispiel dafür dient das Interface `Writer`

```
public interface Writer {  
  
    public final int EOL = '\n';  
    public void open();  
    public void close();  
    public void write(char ch);  
}
```





Vererbung und Interfaces

- Klasse TextBox ist Unterklasse von GUIObject und implementiert das Interface Writer.

```
public class GUIObject {  
    int x, y, width, height;  
    public void resize(int w, int h) { /* ... */ }  
}
```



```
public class TextBox extends GUIObject implements Writer {  
    StringBuilder buffer;  
    String text;  
    public void open() { buffer = new StringBuilder(); }  
    public void close() { text = buffer.toString(); }  
    public void write(char ch) { buffer.append(ch); }  
}
```





Vererbung und Interfaces



```
public static void main(String[] args) {  
    Writer w;           // Deklaration einer Variablen vom Typ Writer  
    w = new TextBox();  // TextBox-Objekt wird angelegt  
    w.open();           // ruft open von TextBox auf (dynamische Bindung)  
    w.write('o');  
    w.write('k');  
    w.close();  
    if (w instanceof TextBox) {  
        TextBox t = (TextBox) w;  
        t.resize(10,20); // Frage: Geht das?  
    }  
    w.resize(10,20); // Frage: Geht das?  
}
```



Interfaces als Typen

- Interfaces sind Typen, d.h. man kann Variablen dieses Typs deklarieren
- Daher können wir `TextBox`-Objekte in einer `Writer`-Variablen speichern und darüber alle `Writer`-Methoden aufrufen.
- Grundlage dafür ist die dynamische Bindung.
- Achtung: Auf die `Writer`-Variable kann nicht die von `GUIObject` geerbte Methode `resize` angewendet werden.



Interfaces vs. Vererbung

- Interfaces dienen dazu, Klassen, die in keiner Vererbungsbeziehung stehen, aber ähnliche Funktionalität anbieten, auf gleiche Weise verwenden zu können.
- Jede Klasse, die das Interface `Writer` implementiert, kann wie ein `Writer` benutzt werden.

```
public class TextBox extends GUIObject implements Writer { /* ... */ }
```



```
public class File implements Writer { /* ... */ }
```



```
public class Buffer implements Writer { /* ... */ }
```





Interfaces vs. Vererbung

```
public static void main(String[] args) {  
    Writer w = new File();  
    w.open();// open von File  
    w = new Buffer();  
    w.open();// open von Buffer  
}
```





Interfaces vs. Vererbung

- Eine Klasse kann nur von einer Klasse erben (Unterklasse sein).
- Eine Klasse kann beliebig viele Interfaces implementieren.

```
class TextBox extends GUIObject implements Writer, Comparable,  
                                              Serializable { /* ... */
```



- TextBox erbt von GUIObject und implementiert die Interfaces Writer, Comparable, Serializable.
- Damit ist TextBox kompatibel zu GUIObject, Writer, Comparable, Serializable.



Interfaces vs. Vererbung

- Das Erben von verschiedenen Oberklassen könnte Schwierigkeiten bereiten
 - Beide Oberklassen könnten Methoden mit gleicher Signatur (Name, Parameter, Rückgabewert) enthalten.
 - Welche Methode würde von der Unterklasse geerbt?
- Das Implementieren von mehreren Interfaces ist unproblematisch
 - Wenn die verschiedenen Interfaces Methoden gleicher Signatur enthalten, implementiert eine Klasse, die beide Interfaces implementiert, genau eine Methode für diese Signatur.
 - Damit ist eindeutig, welcher Code beim Aufruf ausgeführt wird.



Mehrfachvererbung

- Java ist eine Sprache, für die die Design-Entscheidung getroffen wurde, dass jede Klasse maximal eine Oberklasse haben darf.
- In anderen Programmiersprachen wird dieses zugelassen (z.B. in C++).
- Mehrfachvererbung: eine Klasse darf von verschiedenen Oberklassen erben.



Erweitern von Interfaces

- Wir können Interfaces erweitern, d.h. um zusätzliche Konstanten, Methoden ergänzen.
- Bei der Erweiterung verwenden wir das Schlüsselwort `extends`.
- Beispiel: wir erweitern das Interface `Writer` zu einem Interface `ReaderWriter` mit einer zusätzlichen Methode `read`.
- Das Interface `ReaderWriter` enthält damit die Methoden `open`, `close`, `write`, `read`.

```
public interface ReaderWriter extends Writer {  
    char read();  
}
```





Interfaces vs. Vererbung: Zusammenfassung

- Vererbung vererbt tatsächlich Code
- Interfaces vererben nichts (Java < 8), sondern beschreiben Schnittstelle
- Erben von mehreren Oberklassen verboten
- Implementieren mehrerer Interfaces erlaubt, da keine Konflikte der Implementierungen auftreten können
- Interfaces haben nichts / wenig mit Vererbung zu tun
 - Naja, bei Java-8 doch!
 - Fast Mehrfachvererbung
 - Schlüsselwort `default`: Standardimplementierung von Methoden des Interface
 - Motivation: Nachträgliche Erweiterung von Interfaces, ohne dass existierende Implementierungen angepasst werden müssen.



Neuerungen in Java 8

- Es besteht die Möglichkeit, in Interfaces neben Konstanten und Signaturen auch eine **Default-Implementierung** von Methoden und statischen Methoden anzugeben.
- Falls eine Klasse mehrere Interfaces implementiert und dabei die Default-Implementierungen nicht überschreibt, kann es zu Mehrdeutigkeiten kommen.
- Häufig werden in solchen Situationen Fehlermeldungen ausgegeben. Genauere Erläuterungen finden Sie in der Java Language Specification Chapter 9.4



Interfaces vs. abstrakte Klassen

- Unterschied zwischen abstrakten Klassen und Interfaces
 - Abstrakte Klassen können Konstruktoren vorhalten, Interfaces nicht.
 - Abstrakte Klassen können Attribute (Instanzvariablen) haben, Interfaces nicht.
- Beide können einige Methoden implementieren und andere nicht.
- Vorteil Interface: Während eine Unterklasse nur von einer Oberklasse erben kann, kann sie mehrere Interfaces implementieren
- Abstrakte Klassen werden wegen ihrer Flexibilität oft während der Entwicklung eingesetzt
 - beim Interface muss bei jeder Änderung der Schnittstelle in allen implementierenden Klassen nachgezogen werden



Programmieren

5. Vererbung

5.5 Wurzelklasse Object



Java Vererbungshierarchie

- Betrachtet man Basisklassen, wie beispielsweise `Integer` oder `String`, fällt auf, dass auch diese Klassen von einer Oberklasse `Object` erben

Class String

```
java.lang.Object  
    java.lang.String
```

All Implemented Interfaces:

```
Serializable, CharSequence, Comparable<String>, Constable, ConstantDesc
```

```
public final class String  
    extends Object  
    implements Serializable, Comparable<String>, CharSequence, Constable, ConstantDesc
```



Java Vererbungshierarchie

- In Java erben alle Klassen von der Klasse `Object`

Class `Object`

`java.lang.Object`

```
public class Object
```

Class `Object` is the root of the class hierarchy. Every class has `Object` as a superclass. All objects, including arrays, implement the methods of this class.

Since:

1.0

See Also:

`Class`



Java Vererbungshierarchie

- `Object` bietet Methoden an, die von allen Unterklassen genutzt werden.
- Die Methoden sind teilweise schon bekannt. Die wichtigsten sind:
- `equals: boolean equals(Object obj)`
 - Überprüft, ob die beiden Objektreferenzen auf das gleiche Objekt zeigen. Die Methode wird meist überschrieben, weil `==` die Gleichheit der Objektreferenzen prüft und `equals` inhaltliche Gleichheit prüfen soll.
- `clone: protected Object clone()`
 - Erstellt eine neue Kopie von dem Objekt und gibt diese zurück.
- `hashCode: int hashCode()`
 - Gibt einen Hashwert zu dem Objekt zurück.
- `toString: String toString()`
 - Gibt eine „textuelle“ Repräsentation des Strings zurück, die aus dem Klassennamen und einer Darstellung des Hashcodes besteht. Es wird empfohlen, auch diese Funktion zu überschreiben.



Java Vererbungshierarchie

- Object kann wie jede andere Oberklasse benutzt werden.
- So kann auch Object als statischer Typ genutzt werden und beispielsweise String als dynamischer Typ

```
Object obj = new String("Hello World");  
System.out.println(obj.toString());
```





Fazit

- Vererbung und Interfaces sind wichtige Konzepte in Java, die bei der Entwicklung größerer Anwendungen eine zentrale Rolle spielen.
- Durch Vererbung und Interfaces ist es möglich, strukturierter und kompakter zu implementieren.
- Datenstrukturen können mit Vererbung und Interfaces sauber gekapselt werden.
- Es muss kein Code kopiert werden, wenn Objekte ähnliche Funktionalität anbieten.
- Funktionalitäten bleiben erhalten und können einfach erweitert werden.